

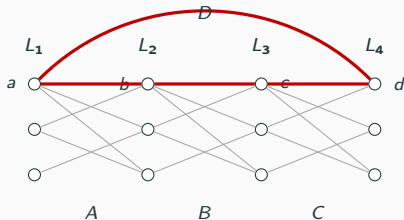
Counting 4-Cycles in Fully Dynamic Graphs

A Cyclic-Join Perspective

Speaker 1 Speaker 2 Speaker 3 Speaker 4

Algorithms seminar – mock PODS 2025

The problem, in one picture



A four-way **cyclic join** on a database:

$$J = A \bowtie B \bowtie C \bowtie D$$

Each binary relation is a bipartite edge set between two attribute domains.

One tuple in J = one 4-cycle in the layered graph.

So $|J|$ is a count of 4-cycles.

What “fully dynamic” actually demands

Edges arrive and leave one at a time. After every update the system must report the exact 4-cycle count.

Cost model: *worst-case* update time

The maximum work on *any single* update – not the average, not the amortised total.

Why worst-case, not amortised?

- Databases need **predictable per-update latency**.
- A 1-second hiccup once every 1000 updates is a production incident, even if the average is fast.
- Stronger guarantee $\Rightarrow$ harder problem.

The target: beat recompute-from-scratch with a per-update *worst-case* guarantee.

The landscape: three patterns on ≤ 4 vertices

Pattern	Best dynamic update time
Triangle (3-cycle)	$O(\sqrt{m})$
4-clique	$O(m)$
4-cycle	$O(m^{2/3})$

- m = number of edges in the host graph.
- Two patterns settle at clean exponents; 4-cycles sit at a weird $2/3$.
- That $2/3$ was the state of the art for over a decade.

Why is the 4-cycle bound stuck at $m^{2/3}$?

Algorithm of Hanauer, Henzinger, Hua (*ICALP 2022*). One-line intuition:

- Split vertices into **low** (degree $\leq m^{1/3}$) and **high** (degree $\geq m^{1/3}$).
- Maintain a *wedge matrix* $W[u, v]$ = number of 2-paths $u-x-v$ with x low-degree.
- Each edge update touches at most $m^{1/3} \cdot m^{1/3} = m^{2/3}$ entries of W .

This $m^{1/3}$ – $m^{2/3}$ split is the **natural ceiling for combinatorial methods**.

Matching combinatorial lower bound from OMv conjecture: $\Omega(m^{1/2-\gamma})$ – doesn't rule out improvement, but the obvious tricks don't help.

The 2025 surprise (Assadi & Shah)

Theorem (Assadi–Shah, PODS 2025)

There is an algorithm maintaining the exact 4-cycle count in worst-case time

$$O(m^{2/3-\varepsilon}), \quad \varepsilon > 0$$

using **fast matrix multiplication** (FMM).

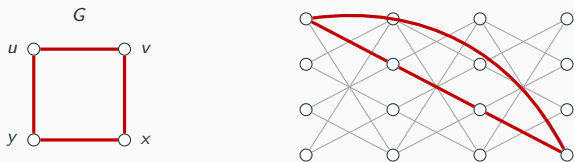
Why is this surprising?

- FMM is a **batch** primitive: multiply two big matrices at once.
- Incremental view maintenance is **streaming**: edges arrive one at a time.
- Folklore said: *FMM cannot give worst-case dynamic improvements.*

Their trick: *phases*. Precompute one matrix product across a batch of $m^{1-\delta}$ updates, while answering queries against the *previous* batch.

Re-presenting through a database lens

Assadi-Shah is dense. Our lens: **IVM for cyclic joins**.



Every general-graph edge $\rightarrow$ **4 layered-graph edges**, one per relation. The architecture (degree classes, biadjacency matrices, where each step lives) becomes *visible*.

Where the FMM step lives

Of the entire machinery, **one product** carries the speedup:

$$A^{L*} \cdot B_{\ell, DD} \quad (\text{rectangular, dimensions } m^{2/3+2\varepsilon} \times m^{1/3-\varepsilon_1+\varepsilon_2})$$

Everything else is bookkeeping:

- degree-class partition (High / Medium / Low; Dense / Sparse),
- chunk-based lazy maintenance ($\sqrt{m^{2/3-\varepsilon_1}}$ updates per chunk),
- phases for the full general-graph algorithm.

All of this exists **to feed that one FMM step a well-conditioned input.**

This is the load-bearing line. The constraint system tells us so.

Closed-form certificate at $\omega = 2$

Assadi–Shah’s feasibility argument: numerical, at the current $\omega < 2.371339$.

Our contribution: closed-form solution at the conjectural optimum $\omega = 2$.

Lemma (Closed-form feasibility at $\omega = 2$)

The constraint system admits the rational solution

$$\varepsilon_1 = \frac{1}{24}, \quad \varepsilon_2 = \frac{5}{24}, \quad \varepsilon = \frac{1}{24}.$$

Five inequalities. Two bind:

- Low-Dense FMM (the ω -dependent one) – **equality**.
- Sparse-Sparse threshold – **equality**.
- The other three: slack.

Why this matters:

- We read off **which** constraint binds.
- It is the FMM one $\Rightarrow$ FMM is *essential*, not incidental.
- Remove FMM $\Rightarrow \varepsilon_1 = 0 \Rightarrow$ bound collapses to $m^{2/3}$.

An empirical check: where does the machinery fire?

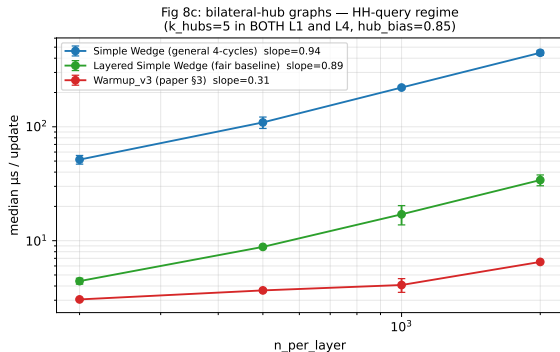
Theory says *FMM is essential*. Practice asks: when does the surrounding routing machinery actually beat a simple baseline?

Setup

- Warm-up algorithm (restricted variant, faithful in shape), NumPy GEMM substituting for FMM.
- Baseline: layer-aware Simple-Wedge, the natural $O(n)$ wedge-counting algorithm on a 4-layered graph.
- Same lifted instances, same update streams.

We are *not* testing asymptotics (NumPy GEMM is n^3 , not n^ω). We are testing the shape of the win: which inputs make the machinery worth its complexity?

The finding: bilateral hubs



The machinery pays off where there are **bilateral hubs** on both sides of the cyclic ordering.

On simpler inputs (unilateral hubs, sparse, no hubs):

- the *wedge baseline already captures most of the benefit*;
- the class-routing layer is overhead.

So the routing machinery is not a generic accelerant – it locates a **specific structural regime** where FMM bandwidth pays for itself.

Takeaway

Two angles converged at the same pressure point:

- **Algebraic** (Lemma): one constraint binds, and it is the FMM one.
- **Empirical**: one input regime fires, and it is the bilateral-hub one.

Both say: the surrounding machinery exists **to feed one well-conditioned FMM step**.

Open directions:

- Push the closed-form bound to the real $\omega \approx 2.371$.
- Other cyclic-join sizes: 5-cycles, 6-cycles, k -cycles.
- Adaptive routing: detect bilateral-hub regimes online, fall back otherwise.

More broadly: **IVM for cyclic joins** is a clean lens on the FMM-vs-streaming tension.

Thank you.

Questions?